CODEBASE AUDIT

hashtag-next

Bugs, design consistency against `globals.css`, flow inconsistencies, unused code and folder structure

Project	hashtag-next (Next.js 16.1.6, React 19.2, Tailwind CSS 4)
Branch / commit	`main` @ `f12ff63` ("payment history response fix")
Date	October 9, 2026
Scope	Every route and component under `src/` (about 31k lines, plus 5.4k lines of committed scratch files), configuration, dependencies and `public/` assets
Prepared by	Claude Code

Contents

1. Summary and how it was checked
2. Fix first
3. A. Architecture, SEO and performance
4. B. Payments, pricing and security
5. C. Broken or fake features
6. D. Design consistency vs `globals.css`
7. E. Flow inconsistencies
8. F. Dead code, dependencies and assets
9. G. Folder structure and naming
10. H. Lint highlights
11. Suggested order

Summary and how it was checked

Every route and component was read, and the claims were backed with tools. Everything ran against a scratch copy of the repository, so the working tree was not modified.

Check	Result
`tsc --noEmit`	Clean. That's only because types are mostly `any` (139 explicit `any` types).
ESLint (project config)	**387 problems: 267 errors, 120 warnings**
`next build`	Passes. It also showed the app server-renders **no content** (see A2).
Compiled `globals.css` with Tailwind 4.3	Confirmed 5 global CSS defects (section D)
Import graph	43 source files never imported, 47 npm dependencies never imported, about 78 MB of unreferenced images

> Items marked **verify backend** depend on how the API behaves, which this audit could not inspect.

Fix first

1. **The global CSS overrides layouts site-wide.** The custom `.container` sits outside any CSS layer, so it beats every Tailwind utility on the same element. Separately, `--container-2xl: 1400px` redefines what `max-w-2xl` means. Together they make all 29 `container max-w-*` layouts ignore their intended width on desktop. For example, the login card (`max-w-md`) renders up to 1280px wide. Details in D1–D2.
2. **The whole app renders blank until the profile API responds**, and any failure of that call logs the user out (A2).
3. **Google Pay and Cash App payment requests are hardcoded to $1.00** (B1).

4. **Pricing is computed per size row, in two conflicting implementations, and sent from the client** (B2–B4).
5. **A shipping label is bought before payment**, and going back and continuing creates another order and label (B5).
6. **Several features fake success:** profile save, the support form, Track Order, the confirmation page, and the PayPal success page prints `page1` (C1–C6).
7. **The login page shows demo credentials**, and registration accepts a 4-character password (B7–B8).

A. Architecture, SEO and performance

A1 `src/contexts/AuthContext.tsx:126` returns `null` while loading. Every page, header and footer sits inside it.

A2 **Verified in the build output:** the prerendered `/about` `<body>` is only `<div hidden=""><!--$--><!--/$--></div>`. The consequences:

- Search engines and link previews see empty pages.
- Logged-in users see a blank screen until `GET user/profile` returns.
- The `catch` at `src/contexts/AuthContext.tsx:57-60` calls `logout()` on any error, so a network blip or a 500 signs the user out.
- `src/contexts/AuthContext.tsx:54` reads `data?.user_profile.mobile` without optional chaining. Accounts with no profile row throw on every load and get logged out.
- **Coupling warning:** this gate currently hides other bugs. Header, Orders and Saved read `localStorage` during render (hydration mismatches), and `HistoryManager` calls `useSearchParams` in the root layout with no Suspense boundary. Both will surface the moment the gate is fixed.

A3 **Login never loads the user.** `await fetchUser()` is commented out (`src/contexts/AuthContext.tsx:100`). After a normal login, the header shows "Account"/"User" and the profile and sidebar are empty until a hard reload.

A4 **Five different "am I logged in" checks:** `useAuth()`, ad-hoc `localStorage.getItem("hastagBillionaire")` (in Header, Orders, Saved, ProductDetail and the customizer), `hasAuthToken()`, the interceptor, and `WishlistContext`. The token key is misspelled ("hastag") and duplicated as a literal in 8 places.

A5 **SEO basics are missing.** Only the root layout exports metadata, so every page is titled "HashtagBillionaire". The product page is a `"use client"` page with no `generateMetadata`. There's no `sitemap.ts` / `robots.ts`, and no `error.tsx`, `global-error.tsx` or `loading.tsx` anywhere. Any render exception falls back to Next's default error screen.

A6 **Performance:**

- `src/components/ProxyImage.tsx:40-51` proxies essentially every remote image through the Node server with `unoptimized` (the "always proxy" list is meaningless because the fallback also returns `true`).
- The customizer re-encodes a PNG data URL on every drag move (`src/components/product/customization/Productcustomizationpage.tsx:1190-1206`) and cache-busts every image load (line 393).
- `antd` is pulled in for three trivial uses (`Spin`, `Badge`, `message`); `framer-motion` for one page.
- Payment history grows `limit` by 3 and re-fetches from page 1 on every scroll, which is quadratic (`src/app/payment-history/page.tsx:88`).
- The home hero image is **6.3 MB**.

A7 `TIMEOUT: 1000000` in `src/constants/config.js:4` is 16.7 minutes, so requests effectively never time out.

B. Payments, pricing and security

B1 **Wallet amount hardcoded.** `src/components/checkout/PaymentSection.tsx:167` and `:171` build `paymentRequest({ total: { amount: "1.00" } })`. The Google Pay sheet shows the buyer $1.00. Cash App Pay ties authorization to the request amount, so real totals are likely to fail.

B2 **Decoration priced per size row, not per design.** See `src/components/product/Addproductconfigurationmodal/Addproductconfigurationmodal.tsx:259-296`:

- Embroidery, 3 sizes × 10 pieces: each row lands in the 1–11 tier and gets its own $35 digitizing fee, totalling **$465**. Priced on the combined 30 pieces it's $300.
- Screen print rows under 12 pieces return `null`, which becomes **$0 decoration**: free printing.

B3 **Two conflicting "single source of truth" pricing tables.**

- `src/components/product/customization/pricing.ts:156-203`: screen print starts at 50 pieces with 2 tiers, and includes `FULL_FRONT`.
- `src/components/product/customization/Productcustomizationpage.tsx:274-345`: screen print starts at 12 pieces with 6 tiers, and `FULL_FRONT` is commented out.
- The `pricing.ts` version is dead code but documented as the canonical one.

B4 **Client-computed prices go to the cart API** verify backend

- `src/components/common/AddToCartModal.tsx:50-86` sends `product_price`, `decoration_price` and `total_price`.
- The ~49% SAGE markup lives in client JS (`src/components/product/customization/Sagequantitypricing.tsx:155`).
- If the server stores these values, anyone can set their own price. The manual-product flow (`src/components/product/manualCartPayload.ts`) does it correctly by sending only IDs; use it as the model.

B5 **Shipping label bought before payment.** `src/components/checkout/CheckoutLayout.tsx:141-157` calls `CreateShipmentLabelApi` right after creating the order. Abandoned checkouts leave purchased labels. "Back to shipping" → Continue creates a second order and label verify backend dedupe

B6 **Square sandbox SDK on every page.** `src/app/layout.tsx:99-102` loads `sandbox.web.squarecdn.com` globally with no `id`. PaymentSection then injects a second SDK, and both define `window.Square`. Remove the layout script.

B7 **Demo credentials on the production login page.** `src/components/login/DemoCredentials.tsx` is rendered at `src/components/login/LoginForm.tsx:138`. `demo123` doesn't even pass the site's own password rules.

B8 **Weak, inconsistent password policy.**

- Registration accepts "Ab1!", which meets 3 of the 4 rules with length optional (`src/components/login/RegisterForm.tsx:101`).
- Change-password requires all 4 rules. The shared `passwordSchema` (`src/lib/validation.ts:49`) is never used.
- Register trims the password (`RegisterForm.tsx:123`) but login doesn't, so passwords with leading or trailing spaces can't log in.

B9 **Image proxy SSRF gaps.** In `src/app/api/proxy-image/route.ts`:

- `fetch` follows redirects, so an allowed host can redirect to `169.254.169.254`.
- IPv6 hostnames come back bracketed (`[::1]`), so the `::1` check never matches.
- There's no DNS check and no response size cap, so it's an open proxy.
- `next.config.ts:15` sets `hostname: "**"`, making `/_next/image` an open optimizer.
- No security headers are set (CSP, `frame-ancestors`), so checkout can be framed.

B10 **Client-set role and committed token.** Every request sends a client-set `role_id: 6` header (`src/api/client.ts:20`); confirm the backend ignores it. An (expired) JWT is committed at `src/api/client.ts:17`. Tokens live in `localStorage`, so any XSS means account takeover.

B11 **Risky dependencies.**

- `@imgly/background-removal` is **AGPL-3.0** (verified in its `LICENSE.md`). That likely needs a commercial license for a proprietary site.
- `squareup@1.0.0` is an unofficial, unused package. Remove it.

C. Broken or fake features

C1 Profile "Save Changes" doesn't save. It shows "Saved" and does nothing (`src/components/account/ProfileSection.tsx:41`). `UpdateUserProfileApi` exists but is never called.

C2 The Support contact form discards messages (`src/components/support/SupportContactForm.tsx:43-51`). Its inputs use a nonexistent `.input` class, so they render with no border. `/support` and `/about` aren't linked from anywhere.

C3 **Track Order is a placeholder:**

- The lookup handler and button are commented out (`src/components/trackorder/TrackLookup.tsx:22-31`).
- `src/components/trackorder/ShipmentDetails.tsx:19-24` shows fabricated **"BlueDart Express"** tracking numbers.
- It reads in-memory `OrdersContext`, which checkout fills with `{ id }` stubs (`src/components/checkout/CheckoutLayout.tsx:265`). `order.orderId.slice` then crashes.

C4 The PayPal success page renders the literal text `page1` (`src/app/checkout/paypal/success/page.tsx:8`).

C5 **No real order confirmation:**

- `SuccessScreen` ignores `orderId`.
- Both payment paths force-redirect to `/categories` after 3 seconds, and the timer isn't cleared.
- `/confirmation` hardcodes `HB00000001` (`src/app/confirmation/page.tsx:32`), lists India-only payment methods (UPI, Cash on Delivery), and nothing links to it.

C6 **PayPal flow:**

- On cancel it still calls the capture API (`src/components/checkout/PaypalReturnStatus.tsx:170`), producing two error toasts.
- The response shape conflicts: `src/api/operations/payment.api.ts:96-101` says `response.data.approve_url`, but the code reads `response.data.data.approval_url` (`src/components/checkout/CheckoutLayout.tsx:302`). One of them is wrong.

C7 Typing a cart quantity fires N sequential +1 API calls; 500 pieces means 499 requests (`src/components/cart/CartItem.tsx:105-142`). `UpdateCartQuantityApi` (`src/api/operations/cart.api.ts:35`) sits unused.

C8 The customizer's wishlist uses the variant ID as the product ID (`src/components/product/customization/Productcustomizationpage.tsx:1128` and `:1731`).

C9 **Orders page:**

- It shows `payment_status`, so paid orders read "SUCCESS" forever and never SHIPPED or DELIVERED (`src/components/orders/OrderCard.tsx:220`).
- Tax is read from the list item instead of the detail (`OrderCard.tsx:535`), so it's likely 0 or NaN.
- Tracking links only appear on Payment History, not on Orders.

C10 **Address form:**

- There's no Address Line 2 input.
- `saving` isn't wired, so Save can be double-clicked into duplicate addresses.
- A new address at checkout auto-selects "the last item in the list" (`src/components/account/AddressSection.tsx:96`), which can be the wrong address.

D. Design consistency vs `globals.css`

D1–D5: defects in `globals.css` itself (confirmed in the compiled CSS)

D1 `--container-2xl: 1400px` (`src/app/globals.css:69`) is Tailwind v4's `max-w-2xl` token, so `max-w-2xl` now means 1400px. It hits the Privacy, Terms and Shipping pages plus FinalCTA, WhatWePrint, SupportHeader and Gallery.

D2 `.container` (`src/app/globals.css:107-141`) is unlayered, so it beats every Tailwind utility on the same element. All 29 `container max-w-*` usages lose their intended width on desktop: the auth cards, Orders, Saved, Checkout, About and others.

D3 **Missing imports.** shadcn's `@import "tw-animate-css"` and `@import "shadcn/tailwind.css"` were dropped. `animate-in`, `fade-in-0`, `zoom-in-95` and `data-open:` / `data-closed:` / `data-checked:` compile to nothing or to selectors Radix never emits (Radix sets `data-state`). Checkbox checked styling and all dialog, sheet, tooltip and accordion animations are dead. `prose` does nothing because `@tailwindcss/typography` was never registered.

D4 **Global overrides.**

- `button { cursor: pointer !important }` beats every `disabled:cursor-not-allowed`.
- `.overflow-y-auto { scrollbar-width: none }` hides scrollbars app-wide.
- Tailwind v4's `dark:` follows the OS setting, so 54 `dark:` classes (in OrderCard, checkout EmptyCart and CheckoutLayout) activate for dark-mode users on a light-only site.

D5 `--color-accent` equals primary, so every `ghost` button hover turns bright yellow.

D6–D10: token drift

D6 **Measured:** about **690 hardcoded hex colors** and about **850 raw palette classes** (`gray-500`, `blue-600`, etc.).

- There are **7 different "primary" yellows**: the token `#FAE52E`, plus `#F5D800`, `#E8D03A`, `#f0c419`, `#CFAF2E`, `#E8CE00` and `#B89000`.
- Off-brand colors include:
  - Green `#2d4a35` on the product gallery and the "View All Products" button.
  - A violet button in checkout's EmptyCart.
  - Blue on the OTP pages and on address selection.
  - Rust `#c8502a` on product cards.
  - Local "tokens" `STAMP` / `THREAD` in CartItem.

D7 **Four parallel CSS-in-JS systems** injected with `<style>`:

- `src/components/categories/categoriesView.styles.ts`, 625 lines, whose header claims "no hardcoded colors" yet it has `background: white` and many `#000` / `#fff` / `rgba` values.
- `src/components/categories/ProductGrid.tsx:31-48`, which defines its own `:root --pg-*` variables and pulls Google's DM Sans in at runtime with `@import`.
- PaymentSection (`ps-*`), which uses DM Sans but never loads it.
- WhatWePrint (`wwp-*`).

D8 **Other mismatches:**

- `font-condensed` is used in 4 checkout files and doesn't exist.
- The customizer offers Montserrat, Dancing Script and Oswald, none of which are loaded, so the preview won't match the print.
- Header and footer use `bg-gray-500/50` instead of tokens.
- There are two toast systems (sonner and antd `message`, even within PasswordSection).
- About 10 hand-rolled modals have no focus trap or Escape handling, while Radix Dialog is used once.
- There are two empty-cart designs: yellow at `/cart`, violet at `/checkout`.
- Card radii vary from `rounded-none` to 24px on the same checkout screen.

D9 **Contrast (computed):**

- Primary yellow on white is **1.29:1** (AA needs 4.5:1), used as text about 40 times (`text-primary`).
- White on yellow appears in the mobile nav and in Track Order's progress steps.
- Checkout grays `#9A9A9A` / `#aaaaaa` are 2.3–2.8:1.

D10 **Inconsistent locale and money formatting:**

- `en-IN` grouping with `$` on the Saved page (`src/app/saved/page.tsx:193`) and in ProductGrid's `formatINR` (`src/components/categories/ProductGrid.tsx:339`) shows "$1,23,456".
- There are 6 different money formatters.

E. Flow inconsistencies

E1 **Product type is decided by matching display titles, differently per page.** ProductDetail checks “Pre-Made products”, “Apparel” and “Promo Products” (and calls them “best-guess placeholders”, `src/app/product/ProductDetail.tsx:651-660`). The customizer checks “Apparel & Uniforms” and “Pre-Made” (`src/components/product/customization/Productcustomizationpage.tsx:697-699`).

E2 **SAGE price differs between pages.** The product page uses `variants[0]`’s server tier, not the selected variant’s (`src/app/product/ProductDetail.tsx:631-634`). The customizer uses meta × 1.4925.

E3 A wishlist toggle or a successful add-to-cart re-fetches the product. That flashes a full-page skeleton and resets the chosen color and size (`src/app/product/ProductDetail.tsx:359` and `:926`).

E4 **Categories:**

- Removing the category pill or clearing the Category facet pushes bare `/categories`. That returns you to the use-case picker and wipes every other filter (`src/components/categories/useCategoriesView.ts:789` together with `:478-520`).
- Every facet click deletes the active search (`clearSearch: true`), so you can’t refine search results.
- The header’s “All Products” lands on the picker, not on all products.
- If the categories API fails, products never load (`useCategoriesView.ts:697`).
- Router navigation runs inside `setState` updaters (`useCategoriesView.ts:806-823` and the other toggles).

E5 **Checkout and auth:**

- Once `pending_order` exists, address and shipping are hidden (`src/components/checkout/CheckoutLayout.tsx:352`). “Back to shipping” then leaves a blank column **verify backend returns pending_order after order creation**
- A resumed pending order shows a hardcoded 8.25% tax estimate.
- The account page’s auth guard is commented out. Payment History shows a raw error when logged out. Orders and Saved show login prompts. The wishlist just toasts “Please login first”.

E6 **Three conflicting artwork upload rules:**

- The customizer accepts any image with no size limit.
- The manual flow accepts only SVG/AI up to 5 MB.
- The product page says “PNG or JPG, max 5MB”.
- Because the manual flow only accepts vectors, the crop dialog and `react-easy-crop` are unreachable.

E7 **Errors and toasts:**

- The interceptor toasts every error, and many callers toast or display it again.
- Login’s “not verified” retry loop can toast up to 5 times.
- The customizer stores the logo as base64 in `sessionStorage`; large logos hit the quota and are silently lost across login.

E8 **Content and history:**

- The footer email is misspelled: `patrick@hashtagbillioniare.net` (`src/components/layout/Footer.tsx:93`).
- Footer labels don’t match their pages: “Refund Policy” opens Shipping & Returns, and “Terms of Service” opens Terms & Conditions.
- `HistoryManager` rewrites browser history behind Next’s router.
- The desktop search box steals focus on every page load (`autoFocus`, `src/components/layout/Header.tsx:202`).

F. Dead code, dependencies and assets

F1 **43 files are never imported**, including:

- All of `Gallery/*` except `GalleryMain`.
- `CheckoutAddressModal` (516 lines), `CategoryFilterBar` (397), `RelatedProducts` (316) and `checkout/Checkout.tsx`.
- Both `AddressCard` components, 4 home sections, 3 order components and 6 `ui/*` primitives.
- `search..api.ts` (a duplicate of `brand.api`), `FabricFacet (1).tsx` (an identical copy), `use-mobile` and `data/checkout.ts`.

F2 **Also effectively dead:**

- `ConatctUs.tsx` (545 lines, commented out of its page).
- `FeaturedProducts` and `common/ProductCard`.
- `OrderedItemCard` (fully commented out).
- `OrdersContext`.
- `TooltipProvider` (0 tooltips).
- 9 unused API functions: `logoutApi` (the server token is never revoked), `UpdateUserProfileApi`, `UpdateCartQuantityApi`, and others.

F3 **Scratch files committed into `src`:**

- `src/app/categories/Untitled-1`, `filter.txt` and `usecase-children`: about 5,100 lines of old CategoriesView copies.
- Two more `Untitled-1.*` files.
- `tsconfig.json:31` explicitly includes two of them.

F4 **47 unused dependencies**, including:

- All 25 `@radix-ui/react-*` packages (the code uses the `radix-ui` meta-package).
- `fabric`, `html2canvas`, `recharts`, `react-square-web-payments-sdk`, `squareup`, `cmdk`, `vaul`, `date-fns`, `next-themes` and `dotenv`.
- `npm`, `install` and `env` as runtime dependencies, plus the `shadcn` CLI.
- `tw-animate-css`, which should be *imported* rather than removed.

F5 **About 78 MB of public assets are unreferenced:**

- `public/assets/images/` (61 MB, full-size duplicates of `GalleryImage`).
- `customization_preview_image/` (9.8 MB; the customizer hardcodes `https://node.hashtagbillionaire.com/...` instead).
- `PrintMethod/` (5.1 MB, including `Gemini_Generated_Image_….png`).
- `logo_header.png`.

F6 Fallback images that don't exist: `/placeholder.png` (CategoryCard, UseCasePickerGate) and `/assets/placeholder.jpg` (ProductGallery).

G. Folder structure and naming

G1 **Inconsistent naming:**

- Typo folder `conatctUs/ConatctUs.tsx`.
- Directory casing is mixed: `Gallery/` and `PaymentHistoryCard/` vs `account/`.
- File casing is mixed: `Facetsection.tsx`, `Sizefacet.tsx`, `Productcustomizationpage.tsx` and `Forgotpasswordclient/` vs PascalCase elsewhere.
- Route names are confusing: `/send-otp` is "forgot password", while `/forgot-password` is "reset password".

G2 **Business logic lives in UI files:**

- Pricing and SAGE helpers are exported from a 3,216-line page component. The product page, the modals and ManualCustomizationPage import from it, and the configuration modal ↔ page import is circular.
- Categories helpers sit in the generic `src/lib/utils.ts`.
- Non-route components live in `app/` (`ProductDetail.tsx`, `CategoriesView.tsx`).

G3 **Duplicated types:**

- `Product` (×5), `CartItem` vs `CartItemType` (bridged with a cast), `ShippingRate`, `SquareMethod`, `ConfiguredVariant`, `Address`, `ProductQueryParams` and `WishlistItem`.
- JavaScript files inside a TypeScript project (`config.js`, `ApiErrorHandler.js`).
- Hook files with no JSX named `.tsx`.

G4 **Docs and config:**

- `README.md` is still the create-next-app boilerplate.
- `project-structure.txt` is stale (it lists files that no longer exist).
- There's no `.env.example`.
- `next.config.ts` uses `module.exports` and carries a stale comment.

H. Lint highlights

Beyond the 139 explicit `any` types and 82 unused variables, ESLint reports:

- Functions used in effects before their `const` declaration (`react-hooks/immutability`).
- Refs read during render (`src/components/categories/useCategoriesView.ts:1175-1240`).
- `Date.now()` called during render in the customizer.
- Ternaries used as statements.
- A `console.log` with `"variants==========>>>"` (`src/components/product/customization/Productcustomizationpage.tsx:672`).
- A `cursor-pointer596` typo (`src/components/login/LoginForm.tsx:123`).
- `export const dynamic` in `"use client"` pages, which is ignored (the build shows `/send-otp` and `/verify-otp` prerendered as static).

Suggested order

1. **One-hour CSS fix:** move `.container` into `@utility`, drop `--container-2xl`, add the two missing imports and the typography plugin, and delete the `button` and `.overflow-y-auto` overrides. Then check each page visually, because several layouts were tuned to the broken widths.
2. Remove the demo credentials, the sandbox script and `page1`. Fix the $1.00 wallet amount and the PayPal field name.
3. Pricing: one shared module, tiers based on total quantity per design, and the server as the authority on price.
4. Auth: render children while loading, only log out on 401, fetch the user after login, and use one `useAuth()` source everywhere.
5. Make the fake features real or remove them: profile save, Support, Track Order, Confirmation.
6. Clean up dead code, dependencies and assets, then do a token pass to replace the hex colors.